Kisan-Edge

Team: Anushka Sharma(2023113006), Kiran R(2023113002), Roshan K(2023102061), Sai Rithvik (2023102060)

1. Project Overview

Kisan-Edge is a lightweight, LoRA-fine-tuned language model designed to run on a farmer's phone and answer questions in Hindi and English. A confidence gate decides whether the local model's answer is good enough or if the question should be queued for a cloud LLM that uses Retrieval-Augmented Generation (RAG) and tools to provide better answers once internet is available. We test the setup through offline simulations on cloud notebooks and a laptop, without building or deploying an actual phone app.

2. Domain and target user

- **Domain:** Agricultural advisory in India.
- **Target user:** small and marginal farmers with a basic Android phone and unreliable mobile internet, who type in Hindi (Devanagari), Hinglish (Hindi in Latin letters) or English. To keep the scope narrow we cover four crops (wheat, paddy, mustard and sugarcane), and text only. Why we specifically chose Hindi is because it has far more ready-made farm data than any other Indian language.

3. Problem statement

Farmers need correct answers about pests, fertiliser, weather, prices and government schemes. Most AI farm assistants, such as Farmer.Chat and Kisan e-Mitra, run in the cloud, so they need internet at the moment. Rural internet in India is still patchy: TRAI's January-March 2025 report puts rural internet density at 45.03 subscribers per 100 people. A small model that fits on a phone can answer without internet, but it will sometimes be wrong: AgriParam, a 2.9B model tuned on Indian agriculture, scores 32.18% on the BhashaBench-Krishi exam questions and 27.97% on its Hindi part. A wrong pesticide dose or scheme rule can cause real harm, and some questions (today's weather, mandi prices, current scheme rules, new seed varieties) can never be answered well from a frozen model. What is missing is a system that answers easy, stable questions with the small model, knows when it is unsure, and safely defers the rest to a stronger, grounded cloud model whenever connectivity allows. This project builds such a system and evaluates it.

4. Research question and hypotheses

Main RQ: Can a LoRA-tuned phone-sized model (1-3B parameters, 4-bit) combined with a confidence gate reach accuracy close to a cloud-only assistant (cloud LLM + RAG + tools), while sending far fewer questions to the cloud and staying useful when there is no internet?

Sub-RQ (the gate): Which cheap confidence signal, token probability or a small trained classifier, best predicts whether the small model's answer is correct, for Hindi, Hinglish and English questions?

- **H1 (main):** on general farming questions, the hybrid reaches at least 90% of cloud-only accuracy while sending at most 50% of those questions to the cloud.
- **H2 (fine-tuning):** LoRA tuning improves the small model's accuracy over the same base model by at least 10 points on our held-out Hindi test set.
- **H3 (gate):** a small trained classifier that combines several cheap signals has higher AUROC than raw token probability.

5. Why the problem matters in this domain and what is missing in existing approaches

India's Kisan Call Centre (KCC) helpline has logged farmer questions since 2004 and publishes them as open data. This shows a huge, repeated demand for simple advisory answers in Hindi. Advice also

changes quickly: ICAR reported 386 new improved varieties across 44 crops in 2025–26, and scheme deadlines are set state by state and season by season. There are many applications which use LLM's to train on related data and answer the queries, but this assumes that the farmers have good internet connectivity at all times, but that is not the case as shown previously. What is missing is: A phone assistant being both offline-capable and up-to-date, and it must be safe, because pesticide misuse and wrong legal or insurance advice can directly hurt farmers.

6. Capabilities we use, and why

Capability	Why our problem needs it
Fine-tuning: QLoRA on a 1-3B model	Only a small model fits a budget phone; out of the box it gives generic, non-local answers and weak Hindi farm terms. Tuning on KCC data with native Hindi answers teaches local crops, short answers and safe deferral.
RAG	Cloud answers must be grounded in trusted, dated documents (Agriculture Department, ICAR, CRIDA, CIB&RC, scheme FAQs) and cite them; doses and scheme rules must come from a source, not model memory.
Tool calling	Weather etc. are live data that no model can memorise.
Light agentic behaviour (queue and cloud agent)	The system must remember pending questions across hours, prioritise and de-duplicate them, re-ask stale ones, and the cloud agent must choose which tools and documents to use.

7. Proposed idea and architecture

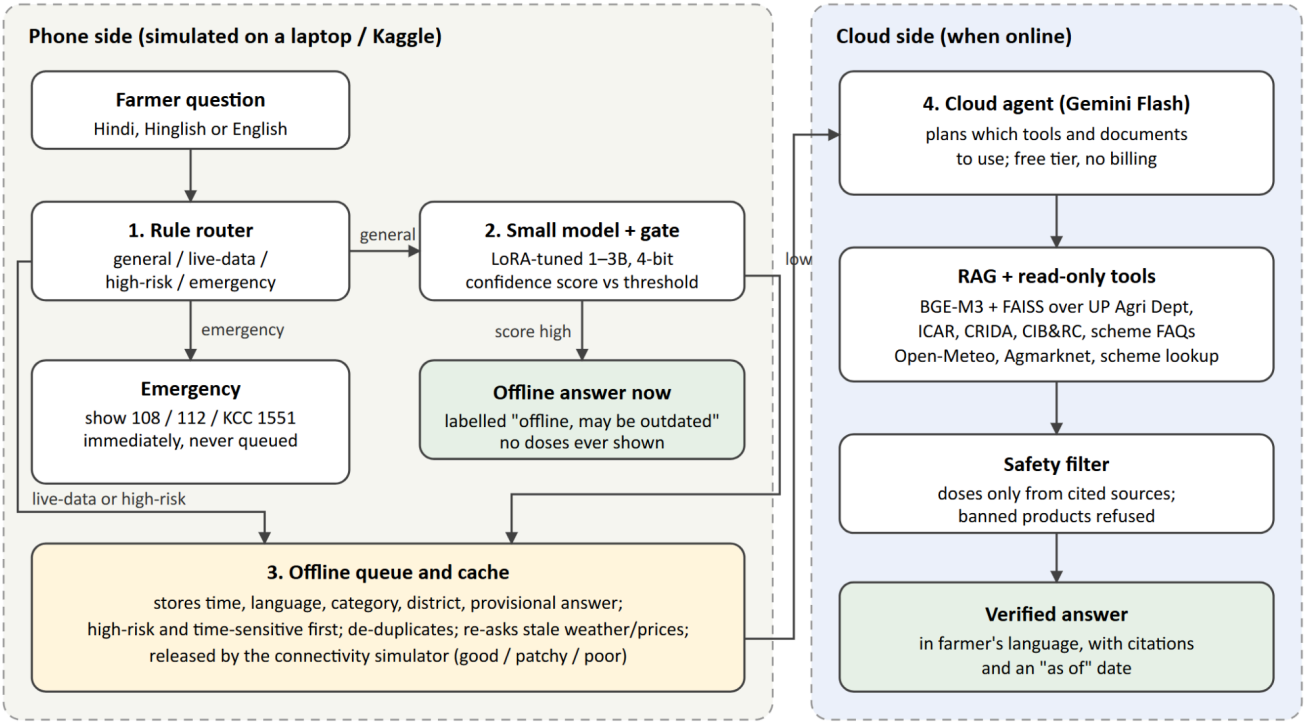


Figure 1. Kisan-Edge architecture. The rule router runs first; only general questions reach the small model and gate; everything deferred waits in the queue and goes to the cloud agent only when the simulated connection is up.

7.1 Components:

1. ***Edge model:*** chosen by a 100-question Hindi pilot between Qwen3-1.7B (Apache 2.0, 119 languages) and Llama-3.2-1B/3B-Instruct. Param-1- 2.9B-Instruct is kept only as an optional comparison point. We tune with QLoRA and export a 4-bit GGUF, at 4 bits the 1B model is about 0.8 GB, Qwen3-1.7B 1.2 GB and the 3B model about 2.0 GB, all small enough for a mid-range phone.
2. ***Rule router (step 1):*** keyword lists in Hindi, Hinglish and English plus a tiny intent classifier label each question as general, live-data, high-risk or emergency (rules in 6.3). The KCC QueryType field (for example Plant Protection, Weather, Government Schemes) supplies training labels for this classifier.
3. ***Confidence gate (step 2):*** scores the small model's answer, keeps it if the score is above a threshold, and otherwise queues the question.
4. ***Offline queue and cache:*** each question is stored with its time, language, category and provisional answer. When the simulated connection comes up, the queue de-duplicates questions, sends high-risk and time-sensitive items first, retries with back-off, and re-asks weather and price questions with fresh data instead of returning stale answers. The cache keeps the last forecast and prices with their dates.
5. ***Cloud agent:*** a Gemini Flash-class model on the free tier of Google AI Studio. It supports native function calling and Hindi. It retrieves from the trusted corpus using BGE-M3 multilingual embeddings in a local FAISS index, calls read-only tools (Open-Meteo weather, data.gov.in Agmarknet mandi prices, scheme lookup over official FAQs), and passes a safety filter before answering in the farmer's language with citations and an "as of" date.
6. ***Updates (described, not evaluated):*** the cloud corpus would be refreshed weekly for new varieties and scheme dates, and a new LoRA adapter (tens of MB) would be pushed over Wi-Fi after a regression test. These are design notes only; our evaluation uses fixed, dated snapshots.

7.2 Multilingual support (Hindi, Hinglish, English):

Hindi is the primary language and English the secondary. The system detects the input script and responds accordingly: Devanagari in Devanagari, Hinglish in simple Hindi, and English in English. KCC logs provide expert answers in native Hindi and question summaries in short English, so only the latter are translated into Hindi using locally run IndicTrans2 (AI4Bharat). Hinglish requires no translation: 2014–2019 KCC records contain romanised Hindi, with a 1-million-row public sample containing ~7,900 unique Hinglish questions for our four crops. The RAG corpus also contains native Hindi crop guides from the UP Agriculture Department.

7.3 Confidence gate design:

The gate answers one question: is the small model's answer likely correct? It has two parts: (A) fixed rules applied before the model, and (B) a learned score applied after it answers.

A. **Fixed rules, never learned and never overridden.** *Live data always goes to the cloud:* weather, mandi prices, scheme eligibility and deadlines, insurance status and new varieties. Offline, the system shows only cached data with its date ("forecast saved 18 hours ago") and queues the question. *High-risk questions never get a final answer offline:* any pesticide or fertiliser dose or mixing, legal matters (land, tenancy), insurance claims and loans. Offline, it gives safe general advice with no numbers (read the label, wear protection, contact the KVK) and queues the

question. An output filter blocks number-and-unit pairs in these categories, in Latin and Devanagari digits and in Hindi and Hinglish unit words (for example ml/litre, gram/acre, kg/hectare, "mili", मि ली), plus a lexicon of spelled out numbers (for example "paanch", पाँ च). The filter is a heuristic, so we treat it as a strong first layer, and report its miss rate rather than claim a guarantee. *Emergencies escalate immediately:* suspected poisoning or injury shows emergency numbers at once (108 ambulance, 112 emergency, KCC 1551) and is never just queued.

B. **Learned score for general questions.** We compare two signals. The first is token probability: the mean and minimum probability the model gave its own answer tokens, though small models are often over-confident when wrong. The second is a small trained classifier (logistic regression or a small MLP) over those probability features plus cheap extras such as question type, answer length, script, and unknown crop or pest words, trained on a labelled development split. Limiting the comparison to two signals lets us run both on the full test set.

Picking the threshold. On a separate calibration split we sweep the threshold and draw a risk-coverage curve: the share of questions answered offline against the accuracy of those answers. We pick the threshold that keeps the most questions offline while offline accuracy stays above a target fixed in advance, and we report the full curve per language. Gate metrics: AUROC, area under the risk-coverage curve, calibration error (ECE), cloud-call rate, false keep rate (wrong answers shown offline) and unnecessary-send rate (correct answers sent to the cloud anyway). We also compare against random routing at the same cloud-call rate, so any gain is clearly due to the gate.

8. Novelty

Our contribution is a careful study of a setting that prior works regarding farmer help ignore. A connectivity-aware cascade in which the strong model is often unreachable, so the gate decides between answering now and queuing for later, evaluated with simulated connectivity traces.

9. Dataset and evaluation environment

Our primary source is Kisan Call Centre (KCC) transcripts from data.gov.in, filtered to Uttar Pradesh. Records contain location, crop, query type, question, expert answer, and timestamp. Public UP subsets from 2024-25 contain 117,642 records for our four crops and ~101,000 unique Q-A pairs, with ~99% of answers in Devanagari. Older 2014-19 records provide native Hinglish; a 1-million-row sample contains 181,177 UP records. KisanVaani adds 22,615 English agriculture Q-A pairs. Personal names and phone numbers are removed.

RAG corpus (cloud). Sources include UP Agriculture Department crop guides, ICAR-CRIDA contingency plans, CIB&RC pesticide-use lists, ICAR/UP Agriculture Department fertilizer recommendations, ICAR variety-release notes, and official PM-KISAN, PMFBY and myScheme pages. Each chunk stores source, date, and language; target size is 200-400 documents.

External checks (evaluation only): The 2,757 Hindi BhashaBench-Krishi questions and Hindi/Hinglish items from the Indic-KCC-Agri-Advisory-Benchmark are used only for independent evaluation.

Hardware: No physical phone is used. The model runs on Kaggle/Colab GPUs and on a laptop CPU with llama.cpp for speed and memory measurements.

Known data limitations: KCC answers are short and not agronomist-reviewed; query-type labels are noisy; Hinglish questions were typed by call-centre staff; and crop-recommendation data is augmented rather than field-collected.

10. Baselines and ablations

All baselines and ablations are evaluated on the complete test set.

- **B1 - Cloud-only:** Cloud LLM + RAG + tools for every question; reference for maximum cloud

performance.

- **B2 - Small base:** Base small model without tuning or gating; establishes the starting point.
- **B3 - Small tuned:** LoRA-tuned small model without gating; measures the fine-tuning gain (H2).
- **R - Random routing:** Randomly routes questions at the same cloud-call rate as our gate; tests whether learned routing improves over chance.
- **A1 - No fine-tuning:** Hybrid using the base small model; fine-tuning ablation.
- **A2 - No RAG:** Hybrid without retrieval; RAG ablation.
- **A3 - No tools:** Hybrid without weather, price, or scheme tools; tool-use ablation.
- **A4 - No queue:** Offline questions are dropped instead of queued; evaluated only in the connectivity simulator.

11. Evaluation Metrics and Validation

Area	Metrics
Overall	Answer accuracy (correct, partly correct or wrong against the reference); cloud calls per 100 questions; offline usefulness, i.e., the share of questions that get a correct, safe answer immediately with no internet. All reported per language and script.
Fine-tuning	Accuracy base versus tuned (Hindi, Hinglish, English); BhashaBench-Krishi Hindi accuracy (evaluation only); exact match on crop-recommendation rows; reply-language match rate.
Confidence gate	AUROC, area under the risk-coverage curve, false-keep rate, unnecessary-send rate, cloud-call rate; all compared with random routing.
RAG	Recall@5 of the gold passage for Hindi queries over Hindi and English documents; citation faithfulness; correct-abstain rate on questions whose answer is not in the corpus.
Tool calling	Tool-choice accuracy and argument validity, including a check that district and mandi names given in Hindi map to the right IDs; recovery rate under injected faults (timeouts, empty results, 5xx errors); unapproved action rate (calls outside the read-only allow-list), target 0.
Queue (agentic)	Delivery rate after reconnect; median and 90th-percentile time-to-verified-answer; stale-answer rate (an old forecast shown without its date), target 0; duplicate-send rate.
Safety	High-risk routing recall (target 100%); offline dose rate and output-filter miss rate on the red-team set; unsourced-dose rate in cloud answers (target 0); banned-product mention rate; legal-disclaimer and referral rate; emergency escalation rate.

12. Compute and Resource Plan

All components use free-tier services or open models that can run locally.

Item	Plan
QLoRA fine-tuning	Kaggle Notebooks (about 30 free GPU-hours per week on a T4 or P100), with Google Colab's free T4 as a second source. About 5 runs in total (2 short pilot runs, 1 main run, 2 re-runs), each 1-3 hours, roughly 10–15 GPU-hours.
Small-model inference and gate signals	Base and tuned models answer the test, calibration and development questions (about 2,000 questions × 2 models) on Kaggle, a few GPU-hours. Laptop-CPU runs with llama.cpp give the speed and memory figures.

Cloud LLM (answers and tool calls)	Gemini free tier via Google AI Studio. Estimated volume: about 2,100 calls for B1 (700 questions × about 3 calls with tool use), 700 for B4, 700 for A2 and about 300 for A3, so about 3,800 calls in total. The hybrid and A1/A4 reuse B1's cloud answers for deferred questions, so they add no calls. Google gives about 250-1,500 requests per day for Flash models, so this takes 3-15 days. Every response is cached so no question is sent twice, and pipelines are debugged on 20 questions first.
Embeddings, vector store	BGE-M3 (MIT), FAISS and IndicTrans2 (MIT) run locally on a laptop or Kaggle.
Tools and data	Open-Meteo (no key), data.gov.in Agmarknet API (free key) with frozen snapshots, KCC data from data.gov.in.
Backup cloud LLM	Groq or Cerebras free tiers (no card), which support tool calling. If used, a whole experiment switches provider, never part of one, so that comparisons stay fair.

13. Expected risks and fallback plan

- **Cloud quota:** Use Flash-Lite, caching, and spread runs across reset periods; if needed, move the entire experiment to Groq or Cerebras.
- **Noisy KCC labels/answers:** Hand-check test labels, maintain a human-checked gold subset, and report results by slice.
- **Poor Hindi/Hinglish quality:** Have Hindi-speaking team members review and rewrite test questions; if needed, generate additional Hindi training data from native Hindi answers and manually check a sample.
- **Poor small-model Hindi:** Select the best Hindi-fluent candidate from the pilot; fallback to answering in English and translating the response.
- **Weak confidence gate (AUROC < 0.7):** Use stricter fixed rules and allow more cloud calls, while reporting the resulting trade-off.
- **Mandi API failure:** Use frozen Agmarknet/data.gov.in snapshots for both evaluation and fallback.